Deliverable 2 - INF201

Kiki Julia Titia Beumer and Thea Ariansen Gathe

Import the required libraries

In [194…
```python
from pathlib import Path
import json
import pandas as pd
import numpy as np
import re
```

Task 1: File ingestion and regular expressions

1. Write a robust file reader function

In [195…
```python
def file_reader() -> tuple[str, str]:
    """Read every file under data/2026 and return (file_text_content, encoding_used)."""

    base_dir = Path.cwd() / "data" / "2026"
    files = sorted(p for p in base_dir.rglob("*") if p.is_file())

    texts: list[str] = []
    encodings: list[str] = []

    for f in files:
        raw = f.read_bytes()  # open each file only once

        try:
            texts.append(raw.decode("utf-8-sig"))  # "utf-8-sig" strips a BOM if present
            encodings.append("utf-8")
        except UnicodeDecodeError:
            texts.append(raw.decode("iso-8859-1"))
            encodings.append("iso-8859-1")

    # Preserve order of first appearance, drop duplicates
    encoding_used = ", ".join(dict.fromkeys(encodings))

    return "\n".join(texts), encoding_used
```

it shows that all files were utf-8 encodinded it is a mix of ascii and utf

2. Write a log parser function that uses regular expressions

In [196…
```python
def parse_log(text: str) -> list[dict]:
    """This function extracts valid sales records from log text, ignoring noise lines."""

    # Define what a valid line should look like
    VALID_LINE = re.compile(
        r"""
        ^\[(?P<timestamp>\d{4}-\d{2}-\d{2}\ \d{2}:\d{2}:\d{2})\]   # [2026-10-15 08:30:00]
        \s+\[(?P<register_id>[A-Za-z0-9_]+)\]                      # [POS_01]
        \s+item=(?P<item>.+?)                                      # Latte (spaces allowed)
        \s+price=(?P<price>\d+(?:\.\d+)?)                          # 4.50
        \s+(?P<currency>[A-Z]{3})                                  # EUR, NOK, USD
        \s+\((?P<status>SUCCESS|REFUND|CANCELLED)\)\s*$            # (SUCCESS)
        """,
        re.VERBOSE,
    )

    # Define list to store the valid sales lines
    valid_sales_lines = []

    # Iterate over each line, store matching lines and remove the noise
    for line in text.splitlines():
        match = VALID_LINE.match(line.strip())

        if not match:
            continue  # skip the noise

        # Store a dictionary of all the named groups from the matches
        record = match.groupdict()

        # Convert the price into a floa
        record["price"] = float(record["price"])

        # Add record to valid_sales_lines list
        valid_sales_lines.append(record)

    return valid_sales_lines
```

Task 2: Recursive utility functions

1. Write a recursive directory search function that finds all log files in a nested directory and returns a list of pathlib.Path objects

In [197…
```python
def find_log_files(directory: str | Path) -> list[Path]:
    """
    Recursively find all log files in a nested directory and return list of pathlib.Path objects
    """

    # Convert the input path to a pathlib.Path object
    path = Path(directory)

    # Initialize empty list to store log files
    log_files = []

    # Iterate through items in sorted order to ensure consistent output
    for item in sorted(path.iterdir()):
        # If an item is a directory, recursively search and add to list
        if item.is_dir():
            log_files.extend(find_log_files(item))
        # If an item is a file with a matching extension, add to list
        elif item.is_file() and item.suffix == '.log':
            log_files.append(item)

    return log_files
```

2. Write a recursive function traverses any combination of nested dictionaries and lists found in this file and returns a set of all register id strings where "active": true.

In [198…
```python
def find_active_registers(node) -> set[str]:
    """
    Recursively collect the ids of all registers with "active": true
    from any combination of nested dicts and lists.
    """

    active = set()

    # Traverse through nested dictionary / list
    if isinstance(node, dict):
        # Check if register dictionary is found
        if 'id' in node and node.get('active') is True:
            active.add(node['id'])
        # Search recursively for all entries in dictionary and add to set
        for value in node.values():
            active |= find_active_registers(value)

    elif isinstance(node, list):
        # If list is found, search recursively for all entries and add to set
        for element in node:
            active |= find_active_registers(element)

    return active


def load_config(file_path) -> dict:
    # Read json file and return dictionary
    with open(Path(file_path), "r", encoding="utf-8") as f:
        return json.load(f)
```

Task 3: Functions for cleaning and analysing data using NumPy

1. Write a function for removing price outliers

In [199…
```python
def remove_price_outliers(df: pd.DataFrame) -> pd.DataFrame:
    """Replace price outliers (more than 3 SD from the mean) with the median
    price and return the cleaned DataFrame."""

    prices = df["price"].astype(float)

    mean_price = prices.mean()
    sd_price = prices.std(ddof=0)  # ddof=0 matches np.std
    median_price = prices.median()

    outliers = (prices - mean_price).abs() > 3 * sd_price
    df.loc[outliers, "price"] = median_price

    return df
```

2. Write a function that computes the element-wise refund ratio

In [200…
```python
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray:
    EPS = 1e-6

    refund_ratio = refund_count / (sales_count + EPS)

    return refund_ratio
```

Task 4: Data analysis using Pandas

1. Write a function that converts the list into a DataFrame, converts the timestamp to datetime (set as index), and sorts the DataFrame chronologically by timestamp.

In [201…
```python
def build_dataframe(transactions: list[dict]) -> pd.DataFrame:
    """
    Converts list of transaction dictionaries into sorted pandas dataframe
    """

    # Convert transaction dictionaries into a dataframe
    df = pd.DataFrame(transactions)

    # Convert timestamp column to datetime
    df['timestamp'] = pd.to_datetime(df['timestamp'])

    # Set timestamp as index and sort chronologically
    df = df.set_index('timestamp').sort_index()

    return df
```

2. Write a function that groups data by item and resamples transactions in 1-hour windows (1h), calculating total price (sum).

In [202…
```python
def group_items(transactions: pd.DataFrame) -> pd.DataFrame:
    """
    Takes a dataframe of transactions, groups data by item and resamples
    transactions in 1h windows, calculating total price
    """

    return transactions.groupby('item')['price'].resample('1h').sum()
```

3. Write a function that creates a pivot table displaying total transaction revenue grouped by item across status categories.

In [203…
```python
def status_total_value(transactions: pd.DataFrame) -> pd.DataFrame:
    """
    Takes a dataframe of transactions, and creates a pivot table displaying
    total transaction revenue grouped by item across status categories
    """

    pivot = transactions.pivot_table(
        values="price",
        index="item",
        columns="status",
        aggfunc="sum",
        fill_value=0
        )

    pivot = pivot.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0)

    return pivot
```

4. Write a function that has the DataFrame as input and calculates the refund ratio per item

In [204…
```python
def refund_ratio(transactions: pd.DataFrame):
    """
    Takes a dataframe of transactions and calculates the refund ratio per item
    using compute_refund_ratio()
    """
    # Get number of transactions per item and status
    counts = (
        transactions.groupby(["item", "status"])
        .size()
        .unstack(fill_value=0)
        .reindex(columns=["SUCCESS", "REFUND", "CANCELLED"], fill_value=0)
    )

    # Make numpy ndarrays of sales and refund count
    sales = counts["SUCCESS"].to_numpy()
    refunds = counts["REFUND"].to_numpy()

    # Return dataframe with sales count, refund count, and ratio per item
    return pd.DataFrame(
        {
            "sales_count": sales,
            "refund_count": refunds,
            "refund_ratio": compute_refund_ratio(sales, refunds),
        },
        index=counts.index,
    )
```

Task 5: Create full workflow

1. Recursively find all log files and extract active register IDs

In [205…
```python
files = find_log_files(Path.cwd() / "data")
config = load_config(Path.cwd() / "data" / "store_config.json")
active_ids = find_active_registers(config)
```

2. Ingest text logs and parse records into a list of dictionaries

In [206…
```python
text, encoding = file_reader()
```

3. Build the initial Pandas DataFrame

In [207…
```python
records = parse_log(text)
df = build_dataframe(records)
```

4. Apply Numpy outlier cleaning to remove 999.00 price glitches

In [208…
```python
cleaned_prices_df = remove_price_outliers(df)
```

5. Apply currency correction: multiply price by 10.0 for all transactions where currency == "NOK"

In [209…
```python
def currency_correction(df: pd.DataFrame) -> pd.DataFrame:
    """Apply currency correction: multiply price by 10 for all transactions
    where currency == "NOK".
    """
    mask = df["currency"] == "NOK"
    df.loc[mask, "price"] = df.loc[mask, "price"] * 10
    return df

currency_corrected_df = currency_correction(cleaned_prices_df)
```

6. Generate resampled reports, pivot tables and refund ratios

In [210…
```python
sampled_reports = group_items(currency_corrected_df)
pivot_tables = status_total_value(currency_corrected_df)
refund_ratios = refund_ratio(currency_corrected_df)
```

7. Save all output files

In [211…
```python
# Cleaned transactions
currency_corrected_df.to_csv("cleaned_transaction.csv", index = False)

# Hourly revenue
sampled_reports.to_json("hourly_revenue.json")

# Status pivot table
pivot_tables.to_csv("status_pivot_table.csv", index = False)

# Item refund ratios
refund_ratios.to_csv("item_refund_ratios.csv", index = False)
```